Chapter 10: Logistic Regression

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

In [1]:
```
!pip install mord
```

```
Requirement already satisfied: mord in /opt/anaconda3/lib/python3.13/site-pa
ckages (0.7)
```

In [2]:
```
import matplotlib.pyplot as plt
import mlba
import numpy as np
import pandas as pd
import seaborn as sns
from mord import LogisticIT # ordinal logistic regression
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.model_selection import train_test_split
%matplotlib inline
```

```
no display found. Using non-interactive Agg backend
```

Example: Acceptance of Personal Loan

The bank's dataset includes data on 5000 customers. The data include the customer's response to the last personal loan campaign (Personal Loan), as well as customer demographic information (Age, Income, etc.) and the customer's relationship with the bank (mortgage, securities account, etc.); see Table 10.1. Among these 5000 customers, only 480 (= 9.6%) accepted the personal loan offered to them in a previous campaign.

The goal is to build a model that identifies customers who are most likely to accept the loan offer in future mailings.

We fit a logistic model to the training set of 3000 Universal Bank customers (test_size=0.4). The outcome variable is Personal Loan, with Yes defined as the success (this is equivalent to setting the outcome variable to 1 for an acceptor and 0 for a nonacceptor).

The positive coefficients for the dummy variables Education_Graduate, Education_Advanced/Professional, and CD Account mean that holding a CD account, and having graduate or professional education (all marked by 1 in the dummy variables) are associated with higher probabilities of accepting the loan offer. In contrast, having a securities account and owning a Universal Bank credit card are associated with lower acceptance rates. For the continuous predictors, positive coefficients indicate that a higher value on that predictor is associated with a higher probability of accepting the loan offer (e.g., Income: higher-income customers tend more to accept the offer). Similarly, negative coefficients indicate that a higher value on that predictor is associated with a lower probability of accepting the loan offer (e.g., Age: older customers are less likely to accept the offer).

In [3]:
```
#Vb67
# loading the dataset
bank_df = mlba.load_data('UniversalBank.csv')
# removeing unwanted columns
bank_df = bank_df.drop(columns=['ID', 'ZIP Code'])

# assume education as categorical, convert to dummy variables
bank_df['Education'] = bank_df['Education'].astype('category')
# converted to categorical variables from numericals
new_categories = {1: 'Undergrad', 2: 'Graduate', 3: 'Advanced/Professional'}
 # renaming the categories in Education
bank_df['Education'] = bank_df.Education.cat.rename_categories(new_categorie
# converting all categorical variables
bank_df = pd.get_dummies(bank_df, prefix_sep='_', drop_first=True, dtype=int

y = bank_df['Personal Loan']
X = bank_df.drop(columns=['Personal Loan'])

# partition data
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.

# fit a logistic regression (set penalty=l2 and C=1e42 to avoid regularizati
logit_reg = LogisticRegression(penalty="l2", C=1e42, solver='liblinear')
logit_reg.fit(train_X, train_y)

# print the intercept
print(f'Intercept {logit_reg.intercept_[0]:.6f}')
# print coefficients
print(pd.DataFrame({'Coeff': logit_reg.coef_[0]}, index=X.columns).transpose

aic_score = mlba.AIC_score(y_true=holdout_y, y_pred=logit_reg.predict(holdou
```

```
                              df=len(train_X.columns)+1)
# print AIC on the holdout (test) set
print(f'AIC {aic_score:.3f}')
```

```
Intercept -12.580956
           Age  Experience    Income    Family     CCAvg  Mortgage  \
Coeff -0.03464    0.036206  0.058893  0.612859  0.240584  0.001013

       Securities Account  CD Account    Online  CreditCard  \
Coeff           -1.018977    3.654862 -0.677481   -0.958366

       Education_Graduate  Education_Advanced/Professional
Coeff            4.204711                         4.356231
AIC -733.998
```

Evaluating Classification Performance

Table 10.3 Propensities for Four Customers in Holdout Data

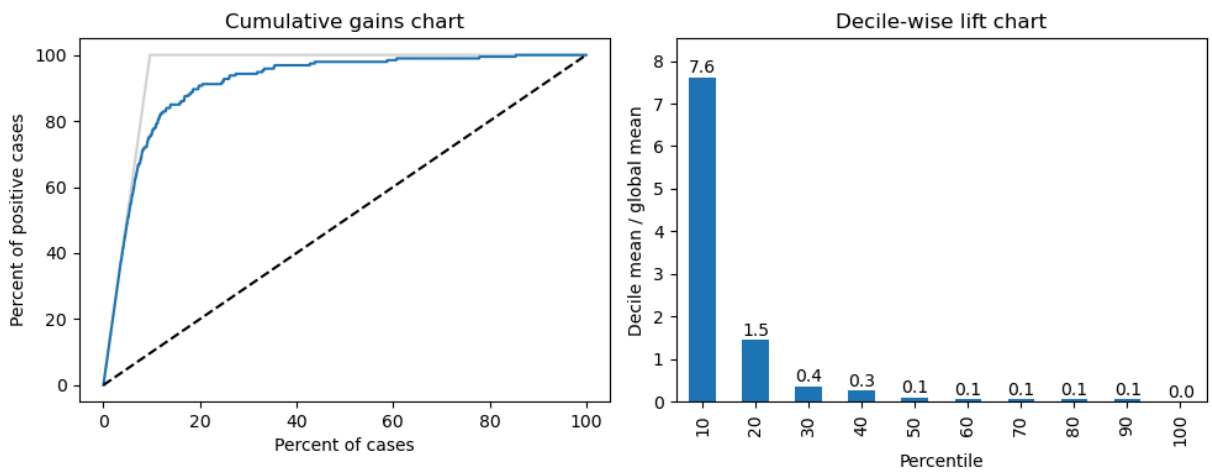
In [4]:
```
#VB67
# Creating a DataFrame with actual labels, predicted probabilities for each
# and the final predicted class from the logistic regression model.

logit_reg_proba = logit_reg.predict_proba(holdout_X)
logit_result = pd.DataFrame({'actual': holdout_y,
                             'p(0)': [p[0] for p in logit_reg_proba],
                             'p(1)': [p[1] for p in logit_reg_proba],
                             'predicted': logit_reg.predict(holdout_X) })

# displaying four different cases
interestingCases = [2764, 932, 2721, 702]
print(logit_result.loc[interestingCases])
```

```
      actual      p(0)      p(1)  predicted
2764       0  0.976153  0.023847          0
932        0  0.331283  0.668717          1
2721       1  0.031373  0.968627          1
702        1  0.985898  0.014102          0
```

Figure 10.3 Cumulative Gains Chart and Decile-wise Lift Chart for the Holdout Data for Universal Bank Loan Offer

In [5]:
```
#Vb67
# Plotting a gains chart and lift chart for the logistic regression results
# using predicted probabilities ranked by class 1

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))
mlba.gainsChart(logit_result, ranking='p(1)', actual='actual', ax=axes[0])
mlba.liftChart(logit_result, ranking='p(1)', actual='actual', ax=axes[1])
```

```
plt.tight_layout()
plt.show()
```



Optional: Ordinal and Nominal Logistic Regression Models

In [6]:
```
#VB67
# Loading accidents dataset and define outcome & predictors

data = mlba.load_data('accidentsFull.csv')
outcome = 'MAX_SEV_IR'
predictors = ['ALCHL_I', 'WEATHER_R']

y = data[outcome]
X = data[predictors]
train_X, train_y = X, y

# Nominal (multinomial) logistic regression

print('Nominal logistic regression')
logit = LogisticRegression(penalty="l2", solver='lbfgs', C=1e24)
logit.fit(X, y)
print(f'  Intercept {logit.intercept_}')
print(f'  Coefficients\n{logit.coef_}')
print()

# Predicting probabilities and store results in a DataFrame
probs = logit.predict_proba(X)
results = pd.DataFrame({
    'actual': y, 'predicted': logit.predict(X),
    'P(0)': [p[0] for p in probs],
    'P(1)': [p[1] for p in probs],
    'P(2)': [p[2] for p in probs],
})
print(results.head().round(3))
print()

# Ordinal logistic regression
print('Ordinal logistic regression')
```

```
logit = LogisticIT(alpha=0)
logit.fit(X, y)
print(f'  Theta {logit.theta_}')
print(f'  Coefficients {logit.coef_}')
print()

# Predicting probabilities and store results in a DataFrame (ordinal model)
probs = logit.predict_proba(X)
results = pd.DataFrame({
    'actual': y, 'predicted': logit.predict(X),
    'P(0)': [p[0] for p in probs],
    'P(1)': [p[1] for p in probs],
    'P(2)': [p[2] for p in probs],
})
print(results.head().round(3))
```

```
Nominal logistic regression
  Intercept [-0.17870851  0.81673248 -0.63802397]
  Coefficients
[[ 0.52491466  0.40469045]
 [ 0.15763357  0.16071816]
 [-0.68254823 -0.56540861]]

   actual  predicted   P(0)   P(1)   P(2)
0       1          1  0.491  0.499  0.010
1       0          0  0.554  0.441  0.004
2       0          0  0.554  0.441  0.004
3       0          1  0.491  0.499  0.010
4       0          1  0.394  0.578  0.028

Ordinal logistic regression
  Theta [-1.06916285  2.77444326]
  Coefficients [-0.40112008 -0.25174207]

   actual  predicted   P(0)   P(1)   P(2)
0       1          1  0.496  0.483  0.021
1       0          0  0.559  0.425  0.017
2       0          0  0.559  0.425  0.017
3       0          1  0.496  0.483  0.021
4       0          1  0.397  0.571  0.031
```

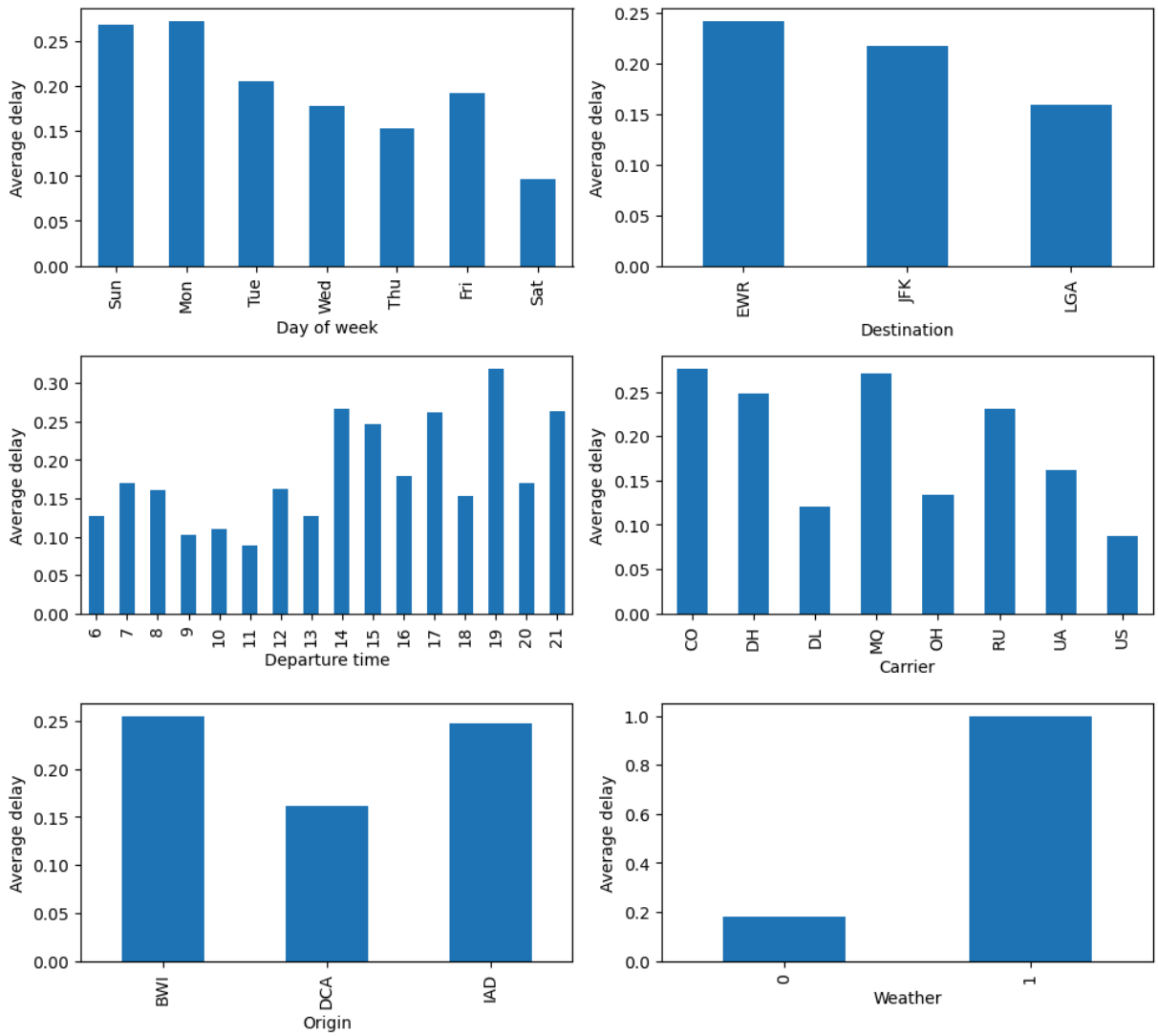
Example of Complete Analysis: Predicting Delayed Flights

In this simplified illustration, we look at six predictors (see Table 10.5). The outcome of interest is whether the flight is delayed or not (delayed means more than 15 minutes late). Our data consist of all flights from the Washington, DC area into the New York City area during January 2004. The percent of delayed flights among these 2201 flights is 19.5%.

EDA - Figure 10.4 Proportion of Delayed Flights by Each of the Six Predictors.

We introduced the column `isDelayed` to be able to easily compute the average delay for each group.

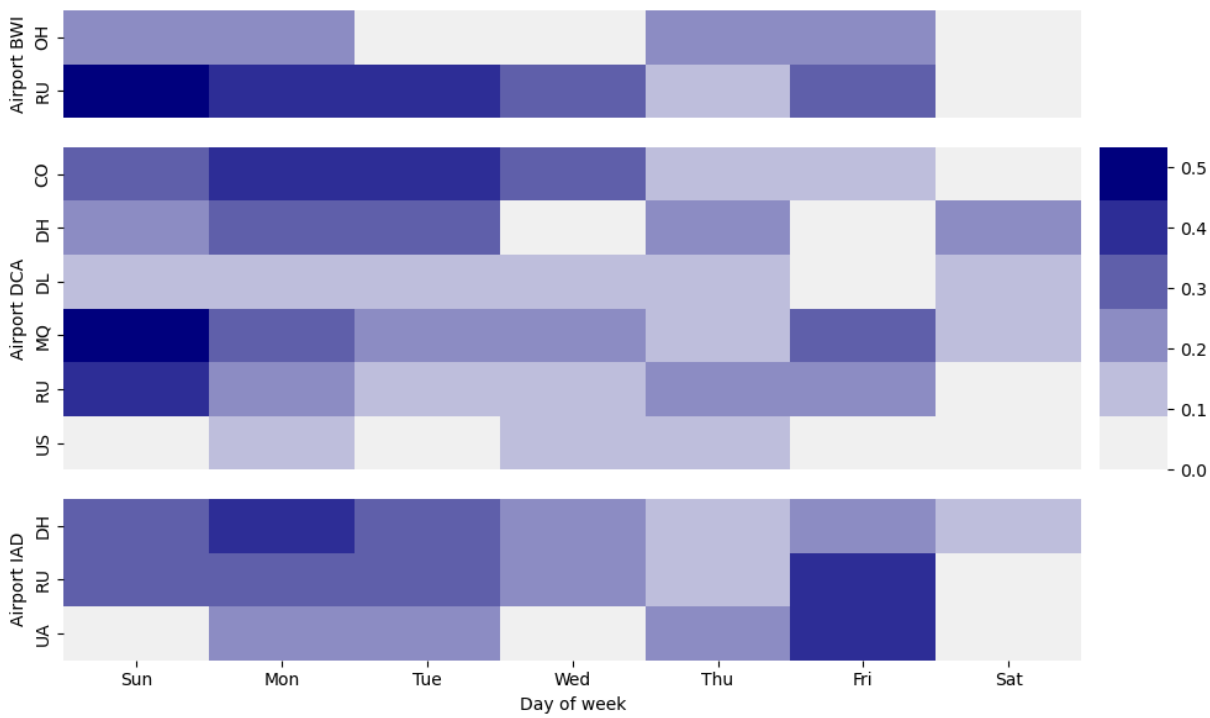
In [15]:
```
#VB67
# Loading the dataset
delays_df = mlba.load_data('FlightDelays.csv')
# Create an indicator variable using Flight Status column
delays_df['isDelayed'] = [1 if status == 'delayed' else 0
                          for status in delays_df['Flight Status']]

# Function to create a bar chart of average delays by a categorical variable
def createGraph(group, xlabel, axis):
    groupAverage = delays_df.groupby([group])['isDelayed'].mean()
    if group == 'DAY_WEEK': # rotate so that display starts on Sunday
        groupAverage = groupAverage.reindex(index=np.roll(groupAverage.index
        groupAverage.index = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat
    # Plot with average delay for the group
    ax = groupAverage.plot.bar(color='C0', ax=axis)
    ax.set_ylabel('Average delay')
    ax.set_xlabel(xlabel)
    return ax
# Function to create a bar chart of average delays by scheduled departure ti
def graphDepartureTime(xlabel, axis):
    temp_df = pd.DataFrame({'CRS_DEP_TIME': delays_df['CRS_DEP_TIME'] // 100
                            'isDelayed': delays_df['isDelayed']})
    groupAverage = temp_df.groupby(['CRS_DEP_TIME'])['isDelayed'].mean()
    ax = groupAverage.plot.bar(color='C0', ax=axis)
    ax.set_xlabel(xlabel); ax.set_ylabel('Average delay')
# Creating a grid of subplots (3 rows, 2 columns) to visualize different del
fig, axes = plt.subplots(nrows=3, ncols=2, figsize=(10, 9))
# Plot delay patterns for different variables
createGraph('DAY_WEEK', 'Day of week', axis=axes[0][0])
createGraph('DEST', 'Destination', axis=axes[0][1])
graphDepartureTime('Departure time', axis=axes[1][0])
createGraph('CARRIER', 'Carrier', axis=axes[1][1])
createGraph('ORIGIN', 'Origin', axis=axes[2][0])
createGraph('Weather', 'Weather', axis=axes[2][1])
#adjusting layouts
plt.tight_layout()
# displaying plots
plt.show()
```



EDA - Figure 10.5 Percent of Delayed Flights (Darker = Higher %Delays) by Day of Week, Origin, and Carrier

In [8]:
```
#VB67
#Group the data to calculate average delay rate by origin, day of week, and

agg = delays_df.groupby(['ORIGIN', 'DAY_WEEK', 'CARRIER']).isDelayed.mean()
agg = agg.reset_index()

# Define the layout of the graph
height_ratios = []
for origin in sorted(delays_df.ORIGIN.unique()):
    height_ratios.append(len(agg[agg.ORIGIN == origin].CARRIER.unique()))

# Define grid layout: 3 rows for 3 airports and 2 columns for heatmap and co
gridspec_kw = {'height_ratios': height_ratios, 'width_ratios': [15, 1]}
fig, axes = plt.subplots(nrows=3, ncols=2, figsize=(10, 6),
                         gridspec_kw = gridspec_kw)
```

```
axes[0, 1].axis('off')
axes[2, 1].axis('off')

maxIsDelay = agg.isDelayed.max() # set consistent color scaling across all h

# Loop over each origin airport to plot a heatmap of dely rates
for i, origin in enumerate(sorted(delays_df.ORIGIN.unique())):
    data = pd.pivot_table(agg[agg.ORIGIN == origin], values='isDelayed', agg
                          index=['CARRIER'], columns=['DAY_WEEK'])
    data = data[[7, 1, 2, 3, 4, 5, 6]]  # Shift last columns to first so wee
    ax = sns.heatmap(data, ax=axes[i][0], vmin=0, vmax=maxIsDelay, # plot he
                     cbar_ax=axes[1][1], cmap=sns.light_palette("navy"))
    ax.set_xticklabels(['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'])
    if i != 2:
        ax.get_xaxis().set_visible(False)
    ax.set_xlabel('Day of week')
    ax.set_ylabel('Airport ' + origin)
plt.tight_layout()
plt.show()
```



Model Fitting

Table 10.7 Estimated Logistic Regression Model for Delayed Flights

By renaming the categories of `Flight Status`, we make sure that the event of interest (delayed) is coded as the second category.

In [9]:
```
#VB67
#loading the dataset
```

```
delays_df = mlba.load_data('FlightDelays.csv')

# converting scheduled departure time to hour bins
delays_df['CRS_DEP_TIME'] = [round(t / 100) for t in delays_df['CRS_DEP_TIME

# converting selected variables to categorical types
delays_df['DAY_WEEK'] = delays_df['DAY_WEEK'].astype('category')
delays_df['CRS_DEP_TIME'] = delays_df['CRS_DEP_TIME'].astype('category')
delays_df['Flight Status'] = delays_df['Flight Status'].astype('category')

# renaming flight status categories for clarity
new_categories = {'ontime': '0-ontime', 'delayed': '1-delayed'}
delays_df['Flight Status'] = delays_df['Flight Status'].cat.rename_categorie

# set predictor and target
predictors = ['DAY_WEEK', 'CRS_DEP_TIME', 'ORIGIN', 'DEST', 'CARRIER', 'Weat
outcome = 'Flight Status'

# creating dummy variables for categorical predictors (drop_first to avoid m
X = pd.get_dummies(delays_df[predictors], drop_first=True)
y = delays_df[outcome]

# spliting into training and validation
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.

# fit logistic regression model with minimal regularization
logit_full = LogisticRegression(penalty="l2", C=1e42, solver='liblinear')
logit_full.fit(train_X, train_y)

# Print the model intercept
print(f'Intercept {logit_full.intercept_[0]:.5f}')
# Print all model coefficients in a readable DataFrame format
with pd.option_context('display.max_columns', None):
    print(pd.DataFrame({'Coeff': logit_full.coef_[0]},
        index=X.columns).transpose().round(3))

# Calculate and print the AIC score for model evaluation
aic_score = mlba.AIC_score(y_true=holdout_y, y_pred=logit_full.predict(holdo
print(f'AIC {aic_score:.3f}')
```

```
Intercept -1.21910
       Weather  DAY_WEEK_2  DAY_WEEK_3  DAY_WEEK_4  DAY_WEEK_5  DAY_WEEK_6
\
Coeff    9.325      -0.598      -0.705      -0.799      -0.296      -1.129

       DAY_WEEK_7  CRS_DEP_TIME_7  CRS_DEP_TIME_8  CRS_DEP_TIME_9  \
Coeff      -0.135           0.631           0.382          -0.365

       CRS_DEP_TIME_10  CRS_DEP_TIME_11  CRS_DEP_TIME_12  CRS_DEP_TIME_13  \
Coeff            0.337            0.078            0.399            0.175

       CRS_DEP_TIME_14  CRS_DEP_TIME_15  CRS_DEP_TIME_16  CRS_DEP_TIME_17  \
Coeff            0.202            1.265            0.628            1.093

       CRS_DEP_TIME_18  CRS_DEP_TIME_19  CRS_DEP_TIME_20  CRS_DEP_TIME_21  \
Coeff            0.285            1.655            1.023            1.077

       ORIGIN_DCA  ORIGIN_IAD  DEST_JFK  DEST_LGA  CARRIER_DH  CARRIER_DL  \
Coeff       -0.01      -0.134    -0.524    -0.546       0.352      -0.685

       CARRIER_MQ  CARRIER_OH  CARRIER_RU  CARRIER_UA  CARRIER_US
Coeff       0.743      -0.711      -0.194       0.315      -0.971
AIC 1004.535
```

Figure 10.6 Confusion Matrix and Cumulative Gains Chart for the Flight Delays Holdout Data using All Predictors

In [10]:

```
#VB67
#Get predicted probabilities and predicted class labels from logistic regres
logit_reg_pred = logit_full.predict_proba(holdout_X)
full_result = pd.DataFrame({'actual': holdout_y,
                            'p(0)': [p[0] for p in logit_reg_pred],
                            'p(1)': [p[1] for p in logit_reg_pred],
                            'predicted': logit_full.predict(holdout_X)})

# Display confusion matrix
mlba.classificationSummary(y_true=full_result['actual'], y_pred=full_result[

# Plotting gains chart to evaluate model performance
mlba.gainsChart(full_result, ranking='p(1)', actual='actual', event_level='1
                figsize=[5, 5])
# Adjusting layout and display plots
plt.tight_layout()
plt.show()
```

```
Confusion Matrix (Accuracy 0.8309)

          Prediction
   Actual  0-ontime 1-delayed
 0-ontime       705         9
1-delayed       140        27
```

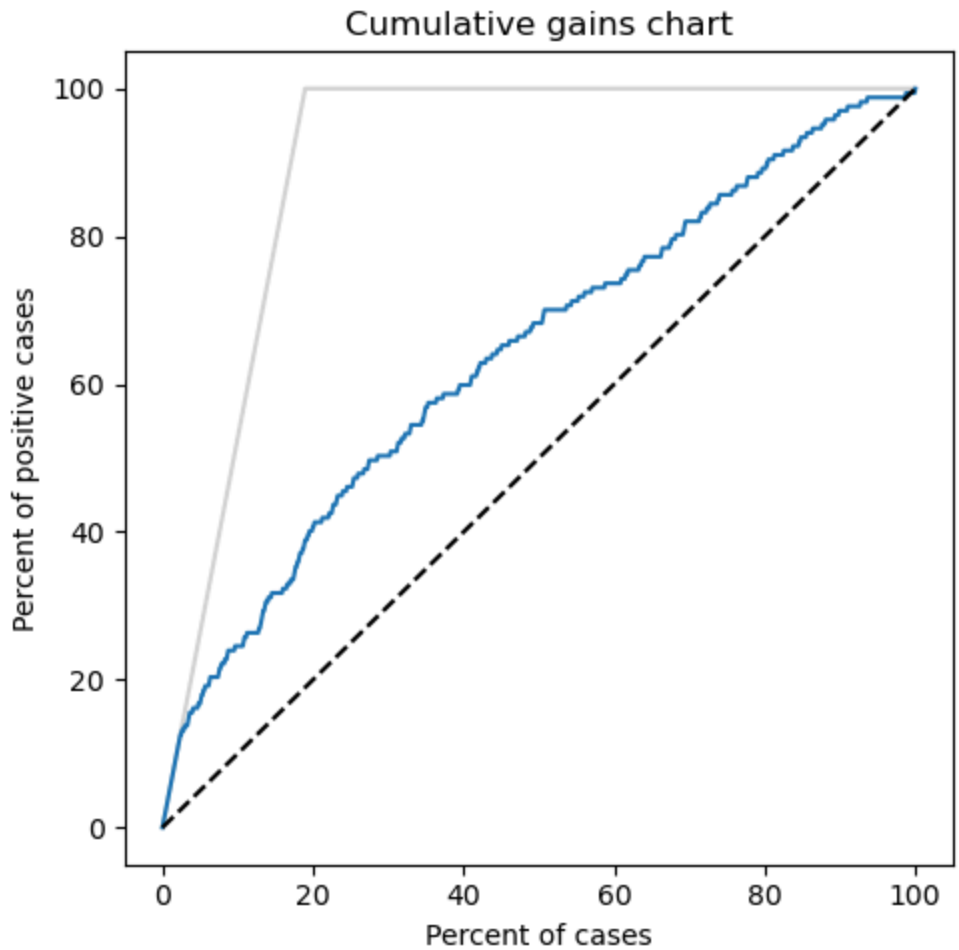


Table 10.9 Logistic Regression Model with Fewer Predictor

First, we find that most carriers depart from a single airport (DCA): for those that depart from all three airports, the delay rates are similar regardless of airport. We therefore drop the departure airport distinction by excluding Origin dummies and find that the model performance and fit are not harmed. We also drop the destination airport for a practical reason: not all carriers fly to all airports. Our model would then be invalid for prediction in nonexistent combinations of carrier and destination airport. We also try grouping carriers, day of week, and hour of day into fewer categories that are more distinguishable with respect to delays. For example, Sundays and Mondays seem to have a similar rate of delays, which differs from the lower rate on Tuesday–Saturday. We therefore group the days of week into Sunday+Monday and Other, resulting in a single dummy variable.

In [11]:
```
#VB67
# manually create reduced set of features
delays_red_df = pd.DataFrame({
    'Sun_Mon' : [1 if d in (1, 7) else 0 for d in delays_df['DAY_WEEK']],
    'Weather' : delays_df['Weather'],
    'CARRIER_CO_MQ_DH_RU' : [1 if d in ("CO", "MQ", "DH", "RU") else 0
```

```
                                for d in delays_df['CARRIER']],
    'MORNING' : [1 if d in (6, 7, 8, 9) else 0 for d in delays_df['CRS_DEP_T
    'NOON' : [1 if d in (10, 11, 12, 13) else 0 for d in delays_df['CRS_DEP_
    'AFTER2P' : [1 if d in (14, 15, 16, 17, 18) else 0 for d in delays_df['C
    'EVENING' : [1 if d in (19, 20) else 0 for d in delays_df['CRS_DEP_TIME'
    'Flight Status': delays_df['Flight Status'],
})

X = delays_red_df.drop(columns=['Flight Status'])
y = delays_red_df['Flight Status']

# spliting into training and validation
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.
                                                          random_state=1)

logit_red = LogisticRegression(penalty="l2", solver='liblinear', C=1e24)
logit_red.fit(train_X, train_y)

print(f'Intercept {logit_red.intercept_[0]:.5f}')
with pd.option_context('display.max_columns', None):
    print(pd.DataFrame({'coeff': logit_red.coef_[0]},
        index=X.columns).transpose().round(3))

aic_score = mlba.AIC_score(y_true=holdout_y, y_pred=logit_red.predict(holdou
                           df=len(train_X.columns) + 1)
print(f'AIC {aic_score:.3f}')

# confusion matrix
mlba.classificationSummary(y_true=holdout_y, y_pred=logit_red.predict(holdou
```

```
Intercept -2.29394
       Sun_Mon  Weather  CARRIER_CO_MQ_DH_RU  MORNING   NOON  AFTER2P  EVENI
NG
coeff     0.59    9.828                1.326   -0.614 -0.698    -0.08    0.5
68
AIC 934.615
Confusion Matrix (Accuracy 0.8343)

          Prediction
   Actual  0-ontime 1-delayed
 0-ontime       711         3
1-delayed       143        24
```

Table 10.10 Regularized Logistic Regression Model for Delayed Flights

We try an automated variable selection approach, by using regularization with L1 penalty (lasso). The 5-fold cross-validation results in a model with 19 non-zero variables.

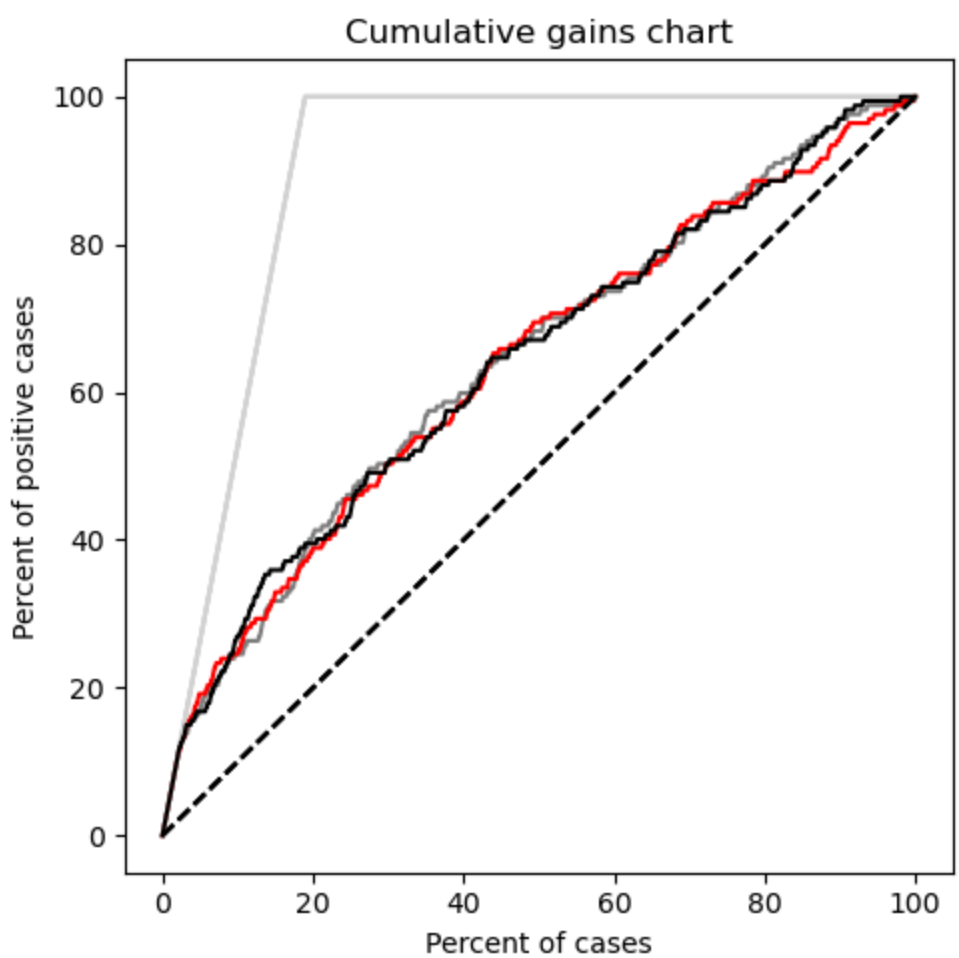
In [12]:
```
#VB67
# Define predictors and outcome variable
predictors = ['DAY_WEEK', 'CRS_DEP_TIME', 'ORIGIN', 'DEST', 'CARRIER', 'Weat
outcome = 'Flight Status'
```

```
# Converting categorical predictors into dummy/indicator variables
X = pd.get_dummies(delays_df[predictors], drop_first=True)
y = delays_df['Flight Status']

# splitting into training and validation
train_X, l1_holdout_X, train_y, l1_holdout_y = train_test_split(X, y, test_s
                                                                random_state=1)
# l1-penalized logistic regression with cross-validation
logit_l1 = LogisticRegressionCV(penalty="l1", solver='liblinear', cv=5)
logit_l1.fit(train_X, train_y)

# Printing model intercept
print(f'Intercept {logit_l1.intercept_[0]:.5f}')
with pd.option_context('display.max_columns', None):
    print(pd.DataFrame({'coeff': logit_l1.coef_[0]}, index=X.columns).transp

# Calculate and print AIC score on holdout data
aic_score = mlba.AIC_score(y_true=holdout_y, y_pred=logit_l1.predict(l1_holc
                           df=len(train_X.columns) + 1)
print(f'AIC {aic_score:.3f}')

# Displaying confusion matrix
mlba.classificationSummary(y_true=holdout_y, y_pred=logit_l1.predict(l1_holc
```

```
Intercept -1.14907
       Weather  DAY_WEEK_2  DAY_WEEK_3  DAY_WEEK_4  DAY_WEEK_5  DAY_WEEK_6
\
coeff    2.321      -0.168      -0.334      -0.456         0.0      -0.725

       DAY_WEEK_7  CRS_DEP_TIME_7  CRS_DEP_TIME_8  CRS_DEP_TIME_9  \
coeff       0.036             0.0             0.0          -0.263

       CRS_DEP_TIME_10  CRS_DEP_TIME_11  CRS_DEP_TIME_12  CRS_DEP_TIME_13  \
coeff              0.0              0.0              0.0           -0.029

       CRS_DEP_TIME_14  CRS_DEP_TIME_15  CRS_DEP_TIME_16  CRS_DEP_TIME_17  \
coeff              0.0            0.582              0.0            0.475

       CRS_DEP_TIME_18  CRS_DEP_TIME_19  CRS_DEP_TIME_20  CRS_DEP_TIME_21  \
coeff              0.0            0.939              0.0            0.413

       ORIGIN_DCA  ORIGIN_IAD  DEST_JFK  DEST_LGA  CARRIER_DH  CARRIER_DL  \
coeff      -0.011         0.0    -0.113    -0.205       0.101        -0.7

       CARRIER_MQ  CARRIER_OH  CARRIER_RU  CARRIER_UA  CARRIER_US
coeff       0.417         0.0         0.0         0.0      -0.995
AIC 1004.535
Confusion Matrix (Accuracy 0.8309)

          Prediction
   Actual  0-ontime 1-delayed
 0-ontime       713         1
1-delayed       148        19
```

Figure 10.7 Cumulative Gains Chart for the Logistic Regression Model with Fewer Predictors and the Lasso Logistic Regression Model on the holdout set.

In [14]:
```
#VB67
# Creating dataframes of predicted probabilities for plotting
red_result = pd.DataFrame({'actual': holdout_y,
        'p(1)': [p[1] for p in logit_red.predict_proba(holdout_X)]})
l1_result = pd.DataFrame({'actual': holdout_y,
        'p(1)': [p[1] for p in logit_l1.predict_proba(l1_holdout_X)]})

# Plotting and compare gains charts of the three models (full, reduced, lass
ax = mlba.gainsChart(full_result, ranking='p(1)', actual='actual',
        event_level='1-delayed', label='Full model', color='grey', figsize=[
mlba.gainsChart(l1_result, ranking='p(1)', actual='actual',
        event_level='1-delayed', label='Lasso model', color='red', ax=ax)
mlba.gainsChart(red_result, ranking='p(1)', actual='actual',
        event_level='1-delayed', label='Reduced model', color='black', ax=ax
#adjusting the layout
plt.tight_layout()
#Display plots
plt.show()
```



In [ ]:

```

```